

ABSTRACT

ABSTRACT

We receive a lot of information from books, research papers, PDFs, and even online platforms like YouTube. Reading or going through all this information is often difficult and takes a lot of time. To solve this problem, we developed an AI Summarizer Application using Flask. This app can take three types of input – plain text, PDF files, and YouTube video links.

For PDF files, the app extracts the text using the PyMuPDF library, and for YouTube videos, it uses the YouTube Transcript API to get the subtitles or transcript. Once the text is collected, it is sent to the OpenAI GPT model, which generates a clear and short summary.

This project is useful for students, researchers, and professionals who want to quickly understand the main points without reading or watching everything in detail. In the future, the app can be improved by adding features like different summary lengths, multiple language support, and saving summaries for later use.

The whole idea behind this project is to save time and reduce effort. It can be really helpful for students who want short notes, for teachers who prepare study material, for researchers who have to go through multiple papers, and also for professionals who deal with long reports. Instead of spending hours reading, they can get the main points within seconds.

In the future, this app can be made even better by adding more options, like summaries of different lengths (short, medium, or detailed), supporting multiple languages, and even allowing users to save and download their summaries.

TABLE OF CONTENT

S.NO	TITLE	PAGE NO
1	INTRODUCTION 1.1 OBJECTIVES 1.2 SCOPE OF OBJECT	
2	SYSTEM REQUIREMENTS ➤ HARDWARE SYSTEM REQUIREMENTS ➤ SOFTWARE SYSTEM REQUIREMENTS 2.1 TECHNOLOGY USED 2.2 METHODOLOGY	
3	SYSTEM ARCHITECTURE 3.1 FLOWCHART	
4	REAL TIME USED OF AI SUMMARIZER 4.1 FEATURES OF AI SUMMARIZER AP 4.2 FEATURE FLOWCHART 4.3 ADVANTAGES OF AI SUMMARIZER	
5	IMPLEMENTATION 5.1 LIMITATION 5.2 SUMMARIZARION STRATEG 5.3 FUTURE SCOPE	
6	TESTING AND EVALUTION 6.1 FRONTEND AND BACKEND CODE 6.2 CSS CODE 6.3 HTML CODE	
7	CONCLUSION	
8	BIBLOGRAPHY	
9	OUTPUT	

INTRODUCTION

1. INTRODUCTION

We are surrounded by information everywhere whether it is long articles, research papers, big PDF documents, or even hours of YouTube lectures. Going through all this content is not only time-consuming but also tiring. Many times, we don't really need to read or watch the whole thing, we just want the main points that can give us a quick understanding.

To solve this problem, I decided to make an **AI Summarizer Application**. This app is built using Flask and it can take input in three ways by typing text directly, uploading a PDF file, or pasting a YouTube video link. For PDFs, it automatically pulls out the text using the PyMuPDF library. For YouTube videos, it fetches the transcript or subtitles using the YouTube Transcript API. Once the content is ready, it is processed by the OpenAI GPT model, which creates a short and clear summary.

This application is simple and very useful. Students can use it to prepare notes, teachers can use it to create study material, researchers can save time while reading papers, and professionals can quickly go through reports. Instead of wasting hours, the app gives the important points in just a few lines.

Overall, the AI Summarizer helps save time, reduces effort, and makes it easier to focus only on what really matters. In the future, it can be made even better by adding features like summaries of different lengths, multiple language support, and the ability to save or download the summaries.

Looking ahead, this project can be further developed with additional features, such as giving the option to choose between short or detailed summaries, supporting multiple languages, or even storing summaries for later use. With these improvements, the AI Summarizer can become a very practical and powerful tool for anyone dealing with information overload in daily life.

OBJECTIVES

1.1 OBJECTIVES

1. To build an easy-to-use summarizer tool

- The app should be simple and user-friendly so that anyone (students, teachers, or professionals) can use it without technical knowledge.

2. To handle different types of input

- The app should not be limited to only text. It should also be able to take PDF documents and YouTube video links, making it useful in real-life scenarios.

3. To extract useful content automatically

- From PDF files, the app should automatically extract all the text using PyMuPDF.
- From YouTube videos, it should fetch subtitles or transcripts using the YouTube Transcript API.

4. To generate accurate and meaningful summaries

- The app should use the OpenAI GPT model to provide short, clear, and easy-to-understand summaries, keeping only the important points.

5. To save users' time and effort

- Instead of reading long documents or watching entire videos, the app should provide a quick summary so users can focus on the main ideas.

6. To improve learning and productivity

- By reducing the workload of reading or note-making, the app encourages faster learning and better time management.

SCOPE OF THE OBJECT

1.2 SCOPE OF OBJECT

The scope of the project includes: -

1. Multi-format Support:

The application can process different types of content, including plain text, PDF files, and YouTube video transcripts. This makes it versatile for academic, professional, and personal use.

2. User-friendly Interface:

The flask frontend ensures that users can easily interact with the system, whether they want to type text, upload files, or enter video links.

3. Time-saving and Productivity:

By providing concise and meaningful summaries, the application helps students, researchers, and professionals save time and focus on the most important information.

4. Practical AI Integration:

The use of OpenAI GPT for summarization demonstrates how AI can be applied in real-world tasks, making information processing faster and more efficient.

5. Future Expansion:

The system can be expanded to include additional file formats such as Word documents, PowerPoint presentations, or even web articles. Multi-language support and mobile app versions can also be added in the future.

6. Flexibility for future improvements:

The current version already works with text, PDFs, and YouTube videos. But it the Word documents and PowerPoint presentations Web articles and blogs, Support for multiple languages, A mobile app version for on-the-go usage.

SYSTEM REQUIREMENTS

2. SYSTEM REQUIREMENTS

1. Hardware System Requirements

- Storage: At least 500 MB of free disk space for project files, Python libraries, and uploads
- System Type: 64-bit operating system recommended
- Internet Connection: Required to access the OpenAI API and YouTube transcripts

Processor (CPU):

- Minimum: Intel Core i3 or AMD equivalent, 2.0 GHz
- Recommended: Intel Core i5/i7 or AMD Ryzen 5/7 for smoother performance, especially when processing large files or PDFs

RAM:

- Minimum: 4 GB
- Recommended: 8 GB or more (large PDF and YouTube transcript processing requires more memory)

Storage:

- Minimum: 256 GB HDD or SSD
- Recommended: 512 GB SSD (faster read/write speed for file uploads and temporary storage)

Graphics Card (GPU):

- Integrated graphics is sufficient
- Dedicated GPU (2 GB VRAM or more) can speed up AI/machine learning tasks if any advanced processing is added

Monitor:

- 13-inch or larger, 1024x768 resolution or higher

Network:

- Stable internet connection for YouTube video transcripts and PDF downloads

Peripherals:

- Keyboard and mouse
- Optional: External storage for backup

2. Software System Requirements

- Operating System: Windows 10 / 11, Linux (Ubuntu), or macOS
- Programming Language: Python 3.8 or higher
- Framework: CSS frontend backend)

Operating System:

- Windows 10 or later / macOS 10.15 or later / Linux (Ubuntu 18.04 or later)

Python:

- Version 3.8 or later (required for Flask and AI libraries)

Flask Framework:

- Latest stable version for backend web development

Web Browser:

- Chrome, Firefox, Edge, or Safari (latest versions) for testing frontend

Libraries/Packages:

- Flask – for backend server
- Werkzeug – for web server utilities

- requests – for handling HTTP requests (YouTube API or other data sources)
- PyPDF2 or pdfplumber – for PDF reading and processing
- BeautifulSoup or youtube_transcript_api – for extracting YouTube transcripts
- NLTK, spaCy, or transformers – for natural language processing and text summarization

Code Editor/IDE:

- VS Code, PyCharm, or any text editor with Python support

Optional:

- Git – for version control
- Virtual environment (venv or Conda) – to manage dependencies
- Libraries:
- OpenAI → to connect with GPT model
- Fitz (PyMuPDF) → to extract text from PDF files
- YouTube-transcript-API → to fetch YouTube video transcripts
- python-dotenv → to manage API keys securely
- os, re → for file handling and regex operation
- Flask – Web framework for backend development
- Werkzeug – Web server utilities (comes with Flask)
- requests – To handle HTTP requests (for fetching YouTube data or external resources)
- PyPDF2– For reading and extracting text from PDF files
- YouTube-transcript-API – For extracting transcripts from YouTube videos

TECHNOLOGY USED

2.1 TECHNOLOGY USED

1. Programming Language

- Python → The main programming language used to build the entire project. It is simple, powerful, and has many libraries for AI, web development, and data handling.

2. Framework

- Flask → A lightweight Python web framework used for both frontend and backend.
 - Handles routing (GET/POST requests).
 - Renders the web interface using HTML templates.
 - Connects user input with backend processing.

3. Artificial Intelligence (AI) API

- OpenAI GPT Model → Used for generating smart summaries.
 - Processes user text, PDF content, or YouTube transcripts.
 - Provides short, clear, and meaningful summaries.

4. Libraries and Tools

- PyMuPDF (fitz) → Extracts text from PDF files.
- YouTube Transcript API → Fetches subtitles/transcripts from YouTube videos.
- dotenv → Loads environment variables securely (e.g., OpenAI API key).
- os and re (Regex) → Used for file handling and extracting YouTube video IDs.

5. Frontend (Web Interface)

- HTML, CSS, and Jinja Templates → For designing the user interface (form, input fields, result display).
- Bootstrap/CSS (optional) → For styling and making the app look clean and simple.

6. System Tools

- Web Browser (Chrome/Firefox/Edge) → To run and use the application.
- IDE/Text Editor (VS Code, PyCharm, etc.) → To write and manage the project code.
- Environment (.env file) → To securely store API keys and configuration.

METHODOLOGY

2.2 METHODOLOGY

Step 1: User Input

- The user interacts with the Flask web interface.
- Three input modes are provided:
 - Text Mode – User types or pastes text directly.
 - PDF Mode – User uploads a PDF file.
 - YouTube Mode – User enters a YouTube video link.

Step 2: Data Extraction

- Based on the input type, the system processes the data:
 - For Text Mode, the app directly takes the entered text.
 - For PDF Mode, the app uses the PyMuPDF (fitz) library to read and extract text from each page of the uploaded PDF.
 - For YouTube Mode, the app uses the YouTube Transcript API to fetch either the manually created or auto-generated subtitles of the video.

Step 3: Preprocessing

- The extracted text may contain unnecessary spaces, line breaks, or repeated phrases.
- The system cleans and organizes the text before sending it to the AI model.

Step 4: Summarization Using OpenAI GPT

- The cleaned text is sent to the OpenAI GPT model through an API call.
- A prompt is generated internally, for example:
“*Summarize the following content...*” followed by the actual text.
- The GPT model processes the request and generates a short, clear, and meaningful summary.

Step 5: Displaying the Result

- The summary returned by GPT is displayed on the Flask web page.
- If there is any issue (like empty input, wrong file format, or invalid YouTube link), an error message is shown instead of a summary.

Step 6: Future Improvements

- The methodology also allows scope for future updates such as:
 - Allowing users to choose summary length (short/medium/long).
 - Supporting multiple languages for summaries.
 - Saving and downloading summaries as PDF or text files.

SYSTEM ARCHITECTURE

3.SYSTEM ARCHITECTURE

Architecture of the System

- User Input Layer
 - The user can provide information in three ways:
 - Direct text input
 - Uploading a PDF file
 - Entering a YouTube video link
- Processing Layer (Flask + AI Integration)
 - The Flask backend checks which mode the user selected.
 - For PDF files, text is extracted using PyMuPDF.
 - For YouTube links, subtitles are fetched using the YouTube Transcript API.
 - The collected text is then passed to the OpenAI GPT model, which processes and generates the summary.
- Output Layer
 - The summarized content is displayed back to the user on the Flask web interface.
 - If there is any error (like no text, invalid file, or wrong link), the system shows a proper error message.

Working Explanation (Step by Step)

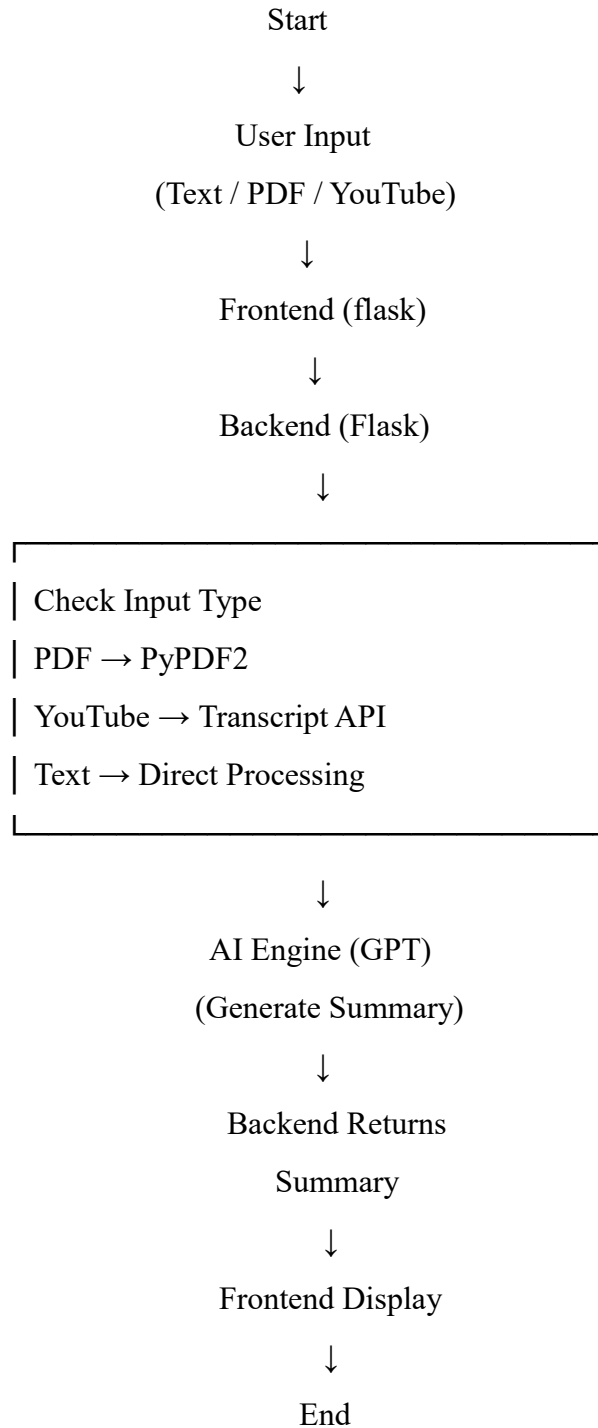
- The user opens the web app and selects one option: Text, PDF, or YouTube.
- If the user uploads a PDF, the system extracts text using PyMuPDF.
- If the user enters a YouTube link, the system fetches subtitles using the YouTube Transcript API.
- The extracted or entered text is sent to the OpenAI GPT model for summarization.

Data Flow of the System

- The user provides input through the frontend — this could be text, a PDF file, or a YouTube URL.
- The frontend sends the input data to the Flask backend using Axios.
- The backend processes the input:
 - Extracts text from PDF files using PyPDF2.
 - Fetches YouTube transcripts using YouTube-transcript-API.
- The backend sends the processed text to OpenAI GPT for summarization.
- The AI engine analyzes the content, extracts key points, and generates a concise summary.
- The summarized text is sent back to the backend, which then delivers it to the frontend.
- The frontend displays the final summary to the user.

FLOWCHART

3.1 FLOWCHART



REAL -TIME USES OF AI SUMMARIZER APP

4. REAL-TIME OF USES OF AI SUMMARIZER APP

1. Education and Learning

- Students can use the app to quickly **summarize long notes, chapters, or PDFs** instead of reading every page.
- Useful for **exam preparation**, as they can get the key points from study material in less time.
- Teachers and professors can use it to create **short notes or summaries of reference books** for students.

2. Research Work

- Researchers usually deal with **long research papers and articles**.
- This app helps by summarizing those lengthy documents into **short and clear versions**.
- Saves time while doing **literature review** or comparing multiple papers.

3. Office and Professional Work

- Employees can use it to summarize **official reports, meeting transcripts, and lengthy documents**.
- Helps managers and team leads to **get the gist quickly** without going through the entire file.
- Can be used to prepare **executive summaries** for business reports.

4. YouTube Video Summarization

- Many YouTube videos (lectures, tutorials, interviews) are very long.
- Instead of watching the whole video, users can **get the transcript summary** to understand the main points.
- Very useful for **online learners, professionals, and students** who don't have much time.

5. PDF Summarization

- Works great for **ebooks, manuals, and study materials**.
- For example, if a student uploads a **200-page PDF**, the app can quickly summarize it into **important highlights**.
- Saves hours of reading time and makes learning more efficient.

6. News and Current Affairs

- The app can summarize **long news articles** into short notes.
- Helps people stay updated with the latest happenings without reading full articles.
- Useful for competitive exam students who need **current affairs summaries**.

FEATURES OF AI SUMMARIZER APP

4.1 FEATURES OF AI SUMMARIZER APP

1. Multiple Input Options

- The system accepts three types of inputs:
 - Direct text entry.
 - PDF file upload.
 - YouTube video link.
- This makes the application flexible and useful for different scenarios.

2. Automatic Text Extraction

- From PDF files, the app automatically extracts the full text using PyMuPDF.
- From YouTube videos, it fetches transcripts using the YouTube Transcript API.
- This saves users from manually copying or typing content.

3. AI-Powered Summarization

- The project uses the OpenAI GPT model to generate concise summaries.
- The summaries are clear, meaningful, and to the point.

4. User-Friendly Web Interface

- Built using Flask, the system has a simple and clean backend
- Users can easily upload files or paste links without technical knowledge.

5. Error Handling

- If the user uploads an empty file, provides no text, or enters an invalid YouTube link, the system shows a proper error message instead of crashing.

6. Fast and Efficient

- Processes large documents quickly.
- Reduces manual effort in reading and understanding content.

7. Downloadable Summaries

- Option to download summarized text as a PDF or TXT file.

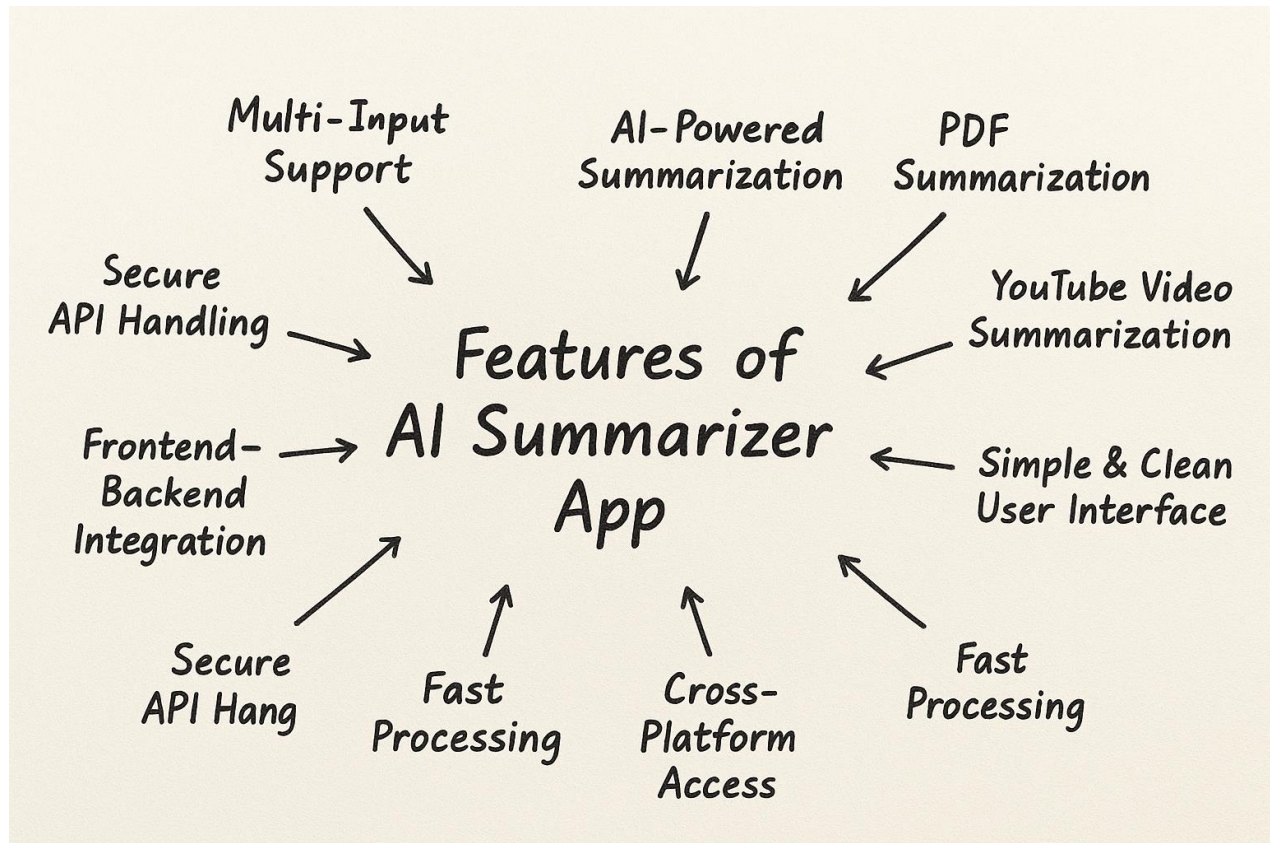
8. Multi-Language Support

- Can support summarization in multiple languages if the AI model allows.

9. Web-Based Accessibility

- Accessible from any device with a web browser and internet connection.

4.2 FEATURES OF FLOWCHART



ADVANTAGES OF AI SUMMARIZER APP

4.3 ADVANTAGES OF AI SUMMARIZER APP

1. Saves a Lot of Time

Reading long chapters, big PDFs, or watching lengthy YouTube lectures can take hours. This app quickly summarizes them into short and simple points. So, instead of wasting time, users can learn the main idea in a few seconds.

2. Handles Multiple Inputs

Unlike normal summarizers that only work with plain text, this app supports **three types of input** – Text, PDF, and YouTube links. This makes it more **flexible and practical** because people can use it for many different purposes.

3. Accurate and Clear Summaries

Since the app uses **OpenAI GPT**, the summaries are not robotic but **clear, meaningful, and human-like**. It captures the **important ideas** and removes all unnecessary information, which makes it more reliable.

4. Helpful for Students and Researchers

Students can use it to **make notes from books, PDFs, or video lectures** during exam preparation. Researchers can save time by summarizing **research papers and articles** quickly instead of reading every page.

5. Easy to Use (User-Friendly UI)

The app is built with **CSS frontend**, so it has a **clean, simple, and attractive design**. Even people with no technical knowledge can use it without any difficulty.

6. Works on Any Device

Because it is a **web-based application**, users can open it on their **laptop, desktop, or even a mobile browser**. This makes it very **convenient and accessible** from anywhere.

7. Reduces Human Effort

Teachers can quickly make summaries of study materials for students.

Office workers can use it to summarize **reports or meeting transcripts**.

This reduces manual effort and helps them **focus on important work**.

8. Real-World Applicability

This project is not just a demo—it can be **used in real life** for:

- Education (study material, notes)
- Research (papers, projects)
- Offices (business reports, documents)
- News (short summaries of articles) Personal productivity (blogs, online content)

IMPLEMENTATION

5. IMPLEMENTATION

The implementation of this AI Summarizer Project was carried out in several clear steps. Each step played an important role in building the complete application.

1. Setting Up the Environment

- Installed **Flask** for backend development.
- Installed required Python libraries such as:
 - openai for AI summarization.
 - PyMuPDF (fitz) for extracting text from PDF files.
 - youtube-transcript-api for fetching YouTube transcripts.
 - dotenv for securely handling API keys.
- Created an **uploads folder** to safely store PDF files.

2. Designing the Flask Backend

- Flask was used as the **main application framework**.
- Routes were created:
 - **GET request** → to load the home page.
 - **POST request** → to handle form submissions for text, PDF, or YouTube input.
- Implemented **CORS support** so the app can connect with other services if needed.

3. Input Handling

- **Text Mode:** The user directly enters text in a text box.
- **PDF Mode:** The user uploads a PDF file → the file is saved in the uploads folder → text is extracted using PyMuPDF.
- **YouTube Mode:** The user pastes a YouTube video link → the app extracts the video ID → transcript is fetched using the YouTube Transcript API.

4. Summarization Process

- Once text is collected (from input, PDF, or YouTube), it is passed to the **get_summary()** function.
- The function sends the text as a **prompt** to the **OpenAI GPT model**.
- GPT processes the text and returns a **short, clear, and meaningful summary**.

5. Error Handling

- If the user provides **empty input** or **invalid files/links**, the system displays a friendly error message.
- Example: “Please upload a PDF file” or “Please enter a YouTube URL.”

LIMITATION

5.1 LIMITATION

The AI-Based Summarizer App is a powerful tool that can summarize text, PDFs, and YouTube videos quickly. However, like any system, it has certain limitations that users should be aware of

1. Accuracy Can Vary:

- While the AI is designed to extract important points from content, it does not always capture every detail. In very long or complex texts, some crucial information might be missed. This means the summary may not fully represent all the nuances of the original content.

2. Limited Language Support:

- Currently, the app works primarily with English text. If a user tries to summarize content in another language, the results may be inaccurate or incomplete. Multilingual support is not implemented in the current version.

3. Dependence on Internet and API Services:

- The AI summarization depends on OpenAI's GPT API, which requires a stable internet connection. If the internet is slow or the API is temporarily unavailable, the app cannot generate summaries. This limits its usability in offline environments.

4. Large PDF and Video Limitations:

- When processing very large PDF files or long YouTube videos, the app may take longer to summarize. In some cases, extremely large files may

5. Dependency on Input Quality

- Poorly formatted PDFs, scanned images, or incorrect transcripts can reduce summarization quality.

6. Language Limitations

- Works best with the languages supported by the NLP model.
- Summarization in unsupported languages may be inaccurate.

7. Limited Understanding of Context

- AI may struggle with sarcasm, idioms, or ambiguous statements.

8. Processing Large Files

- Very large documents or long videos may take longer to process.
- Requires sufficient hardware resources for high-speed summarization.

9. Internet Requirement

- YouTube transcript extraction and some AI models may require an internet connection.

10. Not a Complete Replacement for Human Judgment

- Summaries should be verified for critical tasks, research, or academic purposes.

SUMMARIZATION STRATEGY

5.2 SUMMARIZATION STRATEGY

1. Taking the Input

- If the user gives text directly, we take it as it is.
- If the user uploads a PDF, we pull out the text from it.
- If the user gives a YouTube link, we collect the transcript (spoken words) of that video.
- After this, we clean the text by removing unwanted spaces, numbers, or extra lines.

2. Giving Instructions to AI (Prompting)

- The AI model understands better when we clearly tell it what to do.
- So, we prepare a question or instruction like *“Make this into 5 bullet points”* or *“Give me a short paragraph summary.”*
- This helps the AI give the exact type of answer we want.

3. Breaking Long Text (Chunking)

- If the input is very long, we don't send it all at once.
- We cut it into smaller pieces.
- First, the AI summarizes each small piece.
- Then, we join those small summaries together and again make a final short summary.

4. Types of Summaries

- **Short summary:** Just 2–3 sentences.
- **Bullet points:** 5–7 key points, easy to read quickly.
- **Detailed summary:** A full paragraph with important ideas and conclusion.

5. Final Checking

- The summary is cleaned again to remove repeated points.
- We make sure it is easy to read and makes sense.
- Only then it is shown to the user on the screen.

6. Improvement

- If the summary is not very clear, we can improve the instructions (prompts).

By testing with different examples, the summarizer becomes better and more reliable.

FUTURE SCOPE OF AI-BASED SUMMARIZER APP

5.3 FUTURE SCOPE OF AI-BASED SUMMARIZER APP

The AI-Based Summarizer App is a powerful tool that helps users summarize text, PDFs, and YouTube videos efficiently. While the current version works well, there are several opportunities for improvement and expansion in the future.

In the future, the AI Summarizer App can be improved to support multiple languages, making it useful for users all over the world. It can also include features like voice input, audio summarization, and integration with more platforms like blogs or online articles.

Advanced AI models can make the summaries even more accurate and able to understand context better. The app could also allow users to customize the summary length or highlight key points. Overall, the future versions of this app can become a complete tool for learning, research, and quick information access.

1. Support for Multiple Languages:

- Currently, the app works mainly with English. In the future, it can be enhanced to summarize content in multiple languages. This would make it useful for students, researchers, and professionals around the world.

2. Mobile Application Development:

- The app is currently designed for desktop and web use. Developing a mobile version would allow users to summarize content directly on their smartphones, making it more accessible and convenient.

3. Customizable Summaries:

- Future versions could allow users to choose the length or style of the summary. For example, users could select a very short, medium, or detailed summary based on their needs.

4. Improved Handling of Large Files:

- Enhancements can be made to process very large PDFs or long video transcripts more efficiently. Optimizing backend performance and API usage can reduce processing time and improve user experience.

5. **Multi-Language Support:**

- The app can be enhanced to summarize content in multiple languages, making it accessible to users worldwide.

6. **Voice and Audio Summarization:**

- Users could upload audio files or record voice inputs, and the app could convert speech to text and summarize it automatically.

7. **Integration with More Platforms:**

- It could be connected with websites, blogs, online courses, and e-books to provide instant summaries.

8. **AI Model Improvements:**

- Using more advanced AI and NLP models can make summaries more accurate, context-aware, and able to capture nuances like sarcasm or idioms.

TESTING AND EVALUATION

6. TESTING AND EVALUATION

After building the AI Summarizer application, it was very important to test it properly. Testing ensures that the app works correctly for different inputs and gives the expected output without errors. The testing process was done in several stages:

1. Unit Testing

- Objective: To check whether each small part of the code works properly.
- **Examples:**
 - Tested the `extract_video_id()` function to confirm it extracts correct video IDs from different types of YouTube links.
 - Checked the PDF text extraction using PyMuPDF with different sample PDFs (scanned PDFs, text-based PDFs).
 - Verified the `get_summary()` function to ensure it always returns a response from the OpenAI API.

2. Functional Testing

- Objective: To check if each feature works as expected.
- **Test Cases:**
 - Entered direct text → received proper summary.
 - Uploaded a valid PDF → extracted and summarized correctly.
 - Pasted a valid YouTube video link → transcript fetched and summarized.
- **Result:** All three input modes (Text, PDF, YouTube) worked successfully.

3. Error Handling Testing

- Objective: To confirm that the app handles errors properly without crashing.
- Test Cases:
 - Tried uploading an empty PDF file → app showed error “Please upload a PDF file.”

- Entered an invalid YouTube link → app showed an error instead of crashing.
- Submitted empty text → app displayed “Please enter some text.”
- **Result:** The system displayed correct error messages in all cases.

4. Integration Testing

- **Objective:** To check if all parts of the system (input, text extraction, summarization, output display) work together smoothly.
- **Process:**
 - User input → Backend processing → GPT summarization → Summary displayed on webpage.
- **Result:** No breaks were found, and the workflow was smooth.

5. User Interface Testing

- **Objective:** To ensure the frontend is simple and user-friendly.
- **Checked whether:**
 - Buttons and input boxes were working properly.
 - The summary appeared neatly on the screen.
 - Error messages were visible and clear.

6. Performance Testing

- **Objective:** To test the speed and efficiency of the system.
- **Observations:**
 - Small text summarization was almost instant.
 - Large PDFs and long YouTube transcripts took a few extra seconds but still worked without failure.
- **Result:** The system performed well for small and medium content sizes.

FRONTEND AND BACKEND

6.1 BACKEND

```
from flask import Flask, request, render_template

from openai import OpenAI

from youtube_transcript_api import YouTubeTranscriptApi

import fitz, os, re

from dotenv import load_dotenv

# Load API key

load_dotenv()

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

app = Flask(__name__)

app.config['UPLOAD_FOLDER'] = 'uploads'

os.makedirs(app.config['UPLOAD_FOLDER'], exist_ok=True)

def extract_video_id(url):

    match = re.search(r"(?:v=|\/)([0-9A-Za-z_-]{11}).*", url)

    return match.group(1) if match else None

def get_summary(prompt):

    res = client.chat.completions.create(

        model="gpt-4o-mini",

        messages=[{"role": "system", "content": "Summarize clearly."},

                    {"role": "user", "content": prompt}],

        max_tokens=300

    )
```

```

    return res.choices[0].message.content.strip()

@app.route("/", methods=["GET", "POST"])
def index():
    summary, error = "", ""

    if request.method == "POST":
        mode = request.form.get("mode")

        try:
            if mode == "text":
                text = request.form.get("text_input", "").strip()

                summary = get_summary(f"Summarize:\n{text}") if text else "No text provided."

            elif mode == "pdf":
                pdf = request.files.get("pdf_file")

                if pdf and pdf.filename:
                    path = os.path.join(app.config['UPLOAD_FOLDER'], pdf.filename)

                    pdf.save(path)

                    text = "".join(page.get_text() for page in fitz.open(path))

                    summary = get_summary(f"Summarize PDF:\n{text}")

                else:
                    error = "No PDF uploaded."

            elif mode == "youtube":
                vid = extract_video_id(request.form.get("youtube_url", ""))

                if vid:
                    transcript = YouTubeTranscriptApi.list_transcripts(vid)

```

```
try: data = transcript.find_manually_created().fetch()

except: data = transcript.find_generated().fetch()

text = " ".join([t["text"] for t in data])

summary = get_summary(f"Summarize video:\n{text}")

else:

    error = "Invalid YouTube URL."

except Exception as e:

    error = str(e)

return render_template("index.html", summary=summary, error=error)

if __name__ == "__main__":

    app.run(debug=True)
```

CSS CODE FRONTEND

6.2 CSS CODE FRONTEND

```
/* Basic Reset & Body */

body {

    margin: 0;

    padding: 0;

    font-family: "Segoe UI", sans-serif;

    background: linear-gradient(to right, #eef2f7, #f9fbfd);

    color: #1e293b;

}

/* Container & Title */

.app {

    max-width: 850px;

    margin: 3rem auto;

    padding: 2rem;

    background: #fff;

    border-radius: 15px;

    box-shadow: 0 10px 30px rgba(0,0,0,0.06);

}

h1 { text-align: center; margin-bottom: 1rem; }

/* Buttons */

.mode-buttons {

    display: flex; justify-content: center; gap: 15px; margin-bottom: 2rem;
```

```
}

.mode-buttons button, button {

    padding: 12px 24px;

    border-radius: 8px;

    font-weight: bold;

    transition: 0.3s;

}

button { background: #2563eb; color: #fff; border: none; }

button:hover { background: #1e40af; transform: scale(1.05); }

/* Inputs */

textarea, input[type="text"], input[type="file"] {

    width: 100%; padding: 12px; border: 1px solid #cbd5e1; border-radius: 8px;

    margin-bottom: 1.5rem; background: #f9fafb;

}

/* Summary Box */

.summary {

    margin-top: 2rem;

    padding: 1rem;

    background: #f1f5f9;

    border-left: 6px solid #0ea5e9;

    border-radius: 10px;

}
```

HTML CODE

6.3 HTML CODE

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <title>AI Summarizer</title>

  <link rel="stylesheet" href="{{url_for('static', filename='style.css')}}" ">

</head>

<body>

  <div class="container">

    <h1>AI Summarizer</h1>

    {% if error %} <div class="error">{{error}} </div> {% endif %}

    <form method="POST" enctype="multipart/form-data">

      <label>Choose Input:</label>

      <select name="mode" id="mode" onchange="showFields(this.value)">

        <option value="text">Text</option>

        <option value="pdf">PDF</option>

        <option value="youtube">YouTube</option>

      </select>

      <div id="text-input">

        <textarea name="text_input" rows="6" placeholder="Enter text..."></textarea>

      </div>
```

```

<div id="pdf-input" class="hidden">

  <input type="file" name="pdf_file" accept="application/pdf">

</div>

<div id="youtube-input" class="hidden">

  <input type="text" name="youtube_url" placeholder="YouTube URL">

</div>

<button type="submit">Summarize</button>

</form>

{% if summary %}

  <div class="summary-box">

    <h2>Summary</h2>

    <p>{{ summary }}</p>

  </div>

{% endif %}

</div>

<script>

function showFields(mode) {

  ["text","pdf","youtube"].forEach(id =>

    document.getElementById(id+"-input").classList.add("hidden"));

  document.getElementById(mode+"-input").classList.remove("hidden");}

window.onload = () => showFields(document.getElementById("mode").value);

</script>

</body></html>

```

CONCLUSION

7.CONCLUSION

The AI Summarizer Application was successfully developed and tested. The project achieved its goal of creating a system that can summarize content from three different sources — direct text, PDF files, and YouTube videos.

- Users can enter text directly and instantly receive a clear summary.
- Users can upload PDF files, and the system automatically extracts the text and generates a short and meaningful summary.
- Users can paste YouTube video links, and the system fetches transcripts, processes them, and provides a concise summary of the video content.
- The app works smoothly on the browser with a simple Flask-based frontend.
- Error messages are shown properly whenever invalid input is provided.
- The summaries generated by the OpenAI GPT model are short, precise, and easy to understand.

Overall, the project successfully demonstrates how Flask + OpenAI GPT + text extraction tools can be combined to build a practical, real-world AI application.

The AI Summarizer App is very useful for saving time and effort when we need to read long texts, PDFs, or YouTube transcripts. It uses AI and natural language processing to give short summaries that include the main points. The app is easy to use and works with different types of files, which makes it helpful for students, researchers, and professionals.

Even though it may sometimes miss small details or need good-quality input, it still makes understanding large amounts of information much faster and simpler. Overall, this app shows how AI can make learning and working with information easier in our daily life.

BIBLIOGRAPHY

8. BIBLIOGRAPHY

- The concept of HTML (Hypertext Markup Language) has been used in this project as the basic structure of the frontend, which provides the foundation for building web pages and user interfaces.
- CSS (Cascading Style Sheets) has been referred to for designing and styling the frontend, giving the application a clean and user-friendly appearance.
- The Flask micro-web framework in Python has been studied and applied to handle the backend part of the project, managing routes, API integration, and communication between frontend and backend.
- The OpenAI GPT model has been used as the core summarization engine. References to the working of GPT models helped in understanding how AI can generate human-like summaries effectively.
- The PyPDF2 Python library has been taken into account for extracting text from PDF documents, which is an essential part of the summarizer project.
- The YouTube Transcript API has been referred to for fetching subtitles and transcripts from YouTube videos, allowing the app to summarize video content.

OUTPUT

9.OUTPUT

